EEE3096S Lecture 19 Activity : Sample Solution

Reflecting on:
Design Simple Peripheral Controller Processor (PCP)

Overview of Solution

There were 5 tasks indicated that you should try to deepen your understanding and practice of designing a simple processor and its interfacing to other peripherals. As mentioned in the lecture introducing this topic, this PCP processor is dedicated to peripheral interfacing, it has no buffers of its own, very little memory; all it does it route data from an input to an output, and can make some basic decisions (those decisions linked to state of inputs and outputs). The processor cannot do particularly advanced calculations, the host processor would need to be responsible for more complex decisions and sending a program to the PCP (it can run only one program at a time, it has no facilities for task switching between threads).

You are recommended to work through the tasks that were given first, before reading over this sample solution; however if you are stuck and confused by what to do or what is expected, then by all means go over the solution to the tasks you are struggling with to get a sense of what is expected. Indeed, it is hoped this practice will not only benefit your understanding of processors, and that not all computer processors are necessarily for the purpose of general processing and calculation; as is the case for the PCP it is a rather task-specific processor but such a thing may well be part of a more general-purpose computer (such as a tablet). Part of this activity is to stimulate discussion for the final lecture of discussing exams scope and strategies for dealing with questions and especially the 'Long Question' in the exam.

EEE3096S Lecture 19 Activity: Sample Solutions

Task 1 Solution: Instruction Set Definition

- **Essential Instructions (based on the sample program):**
  - `SETIN` - Sets a specified port as an input port. (Yes, this processor has a whole lot of 'tristate' IO pins, which can be set as either an input or an output … it is up to the designer that is incorporating the processor into a host system to ensure there are no cases where an output links to an output … that could cause a meltdown; input to input is generally not a problem, e.g. multiple devices reading pings on shared bus).
  - `SETOUT` - Sets a specified port as an output port.
  - `WAITIN` - Waits for input on an input line.
  - `IN` - Reads data from a specified input port into a register. These are 8-bit ports, although not all the pins necessarily need to connect to anything.
  - `OUTZ` - Outputs a constant value (in the example given, it was constant 0, the # emphasises a constant) to a specified port *only if* a condition is met (for Z this implies zero flag is set, e.g. for previous instruction the ALU returned zero).
  - `OUTNZ` - Outputs a constant value (in the given example, #1) to a specified port if a condition is met (non-zero flag).
  - `AND` - Performs a bitwise AND operation between two registers.
  - `SHR` - Shifts the bits in a register to the right.
  - `JNZ` - Jumps to a specified label if a condition is met (non-zero flag).
  - `J` - Unconditionally jumps to a specified label.
- **Additional Instructions (for enhanced functionality):**
  - `ADD` - Adds the values of two registers (and possibly save to a third register).
  - `SUB` - Subtracts the value of one register from another.

- `LOAD` - Loads a constant value into a register. (This instruction is often called MOV instead as it is moving a value as opposed to load a value form memory, as the constant is in the simple processor can only be in the instruction).
- `STOREM` - Stores the value (byte) of a register into memory e.g. a small scratch area, some additional registers, perhaps further from the CPU than the main registers. Maybe connects to external memory, if it is decided to actually give the processor some external RAM, but to keep things simply might not want that or interfacing to external memory.
- `LOADM` - loads a byte from scratch memory. Could utilize the STOREM and LOADM for a small stack to enable calling functions.
- `CMP` - Compares the values of two registers.
- `NOP` - Does nothing.

Task 2 Solution: Instruction Encoding

The following encoding scheme for instructions could be applied, this is a fixed-length instruction format to keep it simple (and more aligned to RISC design):

- **Opcode:** 6 bits (allows for up to 64 instructions)
- **Operand 1:** 5 bits (destination register or memory address)
- **Operand 2:** 5 bits (source register, destination address, or immediate value)

A 16-bit instruction length is suggested. It would support 64 instructions, quite a few, but some of them may just be just different alternates of the same time of operation (e.g. MOV immediate value to register, or MOV register to immediate value … the immediate value could).

Diagrammatically this can be illustrated as follows (a diagram is often used in datasheets, and is a good idea for test answer):

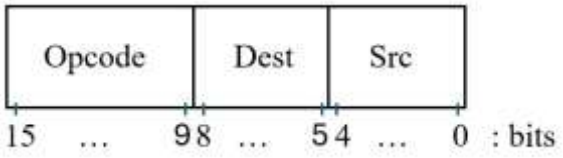

Figure 1: Diagram of PCP istruction format

Task 3 Solution: Processor Block Diagram

The CPU design can be largely along the lines as per the explanations given in L11 and 12, where you have components such as ALU, Control Unit, etc. See diagram on next page.

- **Instruction Register (IR):** Stores instruction from program memory to execute
- **Control Unit:** Fetches, decodes, and executes instructions.
  - *Instruction Decoder:* This can be considered within the CU, it translates instructions into control signals.
- **Register set:** Stores register data used in computations.
- **ALU:** Performs arithmetic and logical operations.
- **Memory Control Unit (MCU) or memory interface:** Handles data transfer between the processor and memory. You will find out soon, in lecture 13, about the MCU. The diagram shows the MCU connecting directly to the register set, that would of course need some intervention form the control unit; in this case accessing memory and IO is via registers.
- **I/O Control Unit (ICU):** Manages data transfer between the processor and external devices.

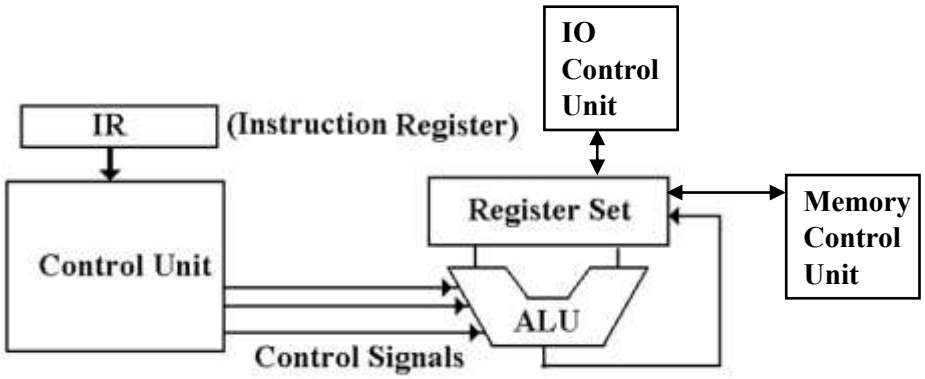

Figure 2: At least a starting point for PCP design

Task 4: Mind Mapping/Connections

You could just sketch ideas and add annotations to links or components to show how they relates, or ideas for how the connections or design aspects would be implemented (I didn't do a mind map in this case). In terms of software to consider, I'd recommend XMind (if using on a PC), there are lots of alternate apps for smartphone or tablet (a decent size tablet is rather useful for this activity).

Task 5: Partial Design Presentation

Having written up responses to Tasks 1-4, the "partial design presentation" is effectively done. The idea (which I didn't clarify so well) is that Tasks 1-4 basically guide your thinking and strategizing for deciding an effective design for PCP … you don't need to worry with writing it up as a more formal document or presenting it (e.g. using slides) to classmates (although that would be a good idea and enhance the activity, bringing in some profcomms, but we don't really have time for that). If you do have any design approaches or want to get my opinion and review on an approach you were considering, then by all means let me know and I'd be happy to review and discuss that.

Conclusion

This concludes the sample solution for the PCP design activity. The solution given here is essentially a small starting point, which students are suggested to look over to evaluate their own designs and consider alternative approaches. We will continue on, the topic of the next set of lectures, to proceed with more complex processor design aspects, in particular dealing with memory including a starting point to what DMA is about and how it is implemented (and why this is such an important part of modern computer systems).